Table des matières

Labo 01 — Premiers pas dans l'entrepôt AdventureWorks DW 1
Partie 1 — Prendre la base en main 1
Partie 2 — Mettre la théorie en pratique 2
Partie 3 — Un regard vers l'OLTP 6
Et ensuite ? 7

Labo 01 — Premiers pas dans l'entrepôt AdventureWorks DW

Laboratoire du Cours 01 — Formation SQL avancé

Votre environnement est déjà en place (Cours 00 : AdventureWorks DW monté dans DuckDB). Ce labo sert à mettre la théorie du Cours 01 en pratique sur cette vraie base, en étoile. Comptez trois quarts d'heure.

Un seul terrain, pour cette fois — Ce cours n'a qu'un laboratoire, sur l'entrepôt. Le double chemin commence au Cours 02, où chaque module a deux labos, un par terrain. La Partie 3 ci-dessous vous en donne un avant-goût, sans rien exécuter.

Tout ce qui suit se tape vraiment, dans la ligne de commande DuckDB. Depuis `Projet_SQL/`, ouvrez l'entrepôt par `duckdb adventureworksdw.duckdb`, puis posez le schéma par `USE dbo;`. Le SQL s'apprend avec les doigts, pas juste avec les yeux.

L'entrepôt n'est pas encore monté ? Retournez au Cours 00, qui le fait en une commande.

Tables du jour — Ce cours interroge `DimProduct`, `DimProductSubcategory`, `DimProductCategory` et `FactInternetSales`, quatre des huit tables montées au Cours 00. Si le `SHOW TABLES` ci-dessous ne les affiche pas, c'est presque toujours que le `USE` manque.

Partie 1 — Prendre la base en main

Théorie : `Theorie_01_SQL.md`, sections 4 et 5.

On explore d'abord. Tapez chaque requête à l'invite, puis `Entrée`. Une requête se termine par un point-virgule ; sans lui, DuckDB attend la suite.

Se placer dans le schéma, puis voir les tables de l'entrepôt :

```
USE dbo;
SHOW TABLES;
```


© 2026 iAlexMG. Tous droits réservés.

Sans le `USE`, le schéma courant est `main`, qui est vide : `SHOW TABLES` ne répond rien et les requêtes suivantes disent que la table n'existe pas. C'est la première ligne de chaque séance.

Regarder les premières lignes de la table des produits :

```sql
SELECT ProductKey, EnglishProductName, Color, ListPrice, ProductSubcategoryKey
FROM dbo.DimProduct
LIMIT 10;
```

Notez que certaines lignes ont `ListPrice` et `ProductSubcategoryKey` vides, à `NULL`, alors que `Color` affiche `NA`. Ce n'est pas la même chose, et c'est tout l'objet des sections 2.4 et 2.5.

Partie 2 — Mettre la théorie en pratique

On reprend, une par une, les idées du Cours 01. Pour chaque étape : l'objectif, la requête à taper, et ce que vous devez observer.

2.1 — `SELECT` et `WHERE` : filtrer des lignes

Théorie : `Theorie_01_SQL.md`, sections 2 et 6.

Objectif : afficher les produits rouges à plus de 1 000 dollars.

```sql
SELECT EnglishProductName, Color, ListPrice
FROM dbo.DimProduct
WHERE Color = 'Red' AND ListPrice > 1000
ORDER BY ListPrice DESC;
```

Observez. Seuls les produits rouges dépassant 1 000 dollars apparaissent, du plus cher au moins cher. Changez `'Red'` en `'Black'` : la liste change.

2.2 — `JOIN`, `GROUP BY` et `HAVING` : la question analytique type

Théorie : `Theorie_01_SQL.md`, sections 6 et 7.

Objectif : les produits les plus vendus en ligne, plus de 200 unités au total.

C'est le geste de l'entrepôt : la table de faits porte les quantités, la dimension porte le nom lisible. On les joint, puis on agrège.

```sql
SELECT p.EnglishProductName,
       SUM(f.OrderQuantity) AS total_vendu
FROM dbo.FactInternetSales AS f
JOIN dbo.DimProduct         AS p ON p.ProductKey = f.ProductKey
GROUP BY p.EnglishProductName
HAVING SUM(f.OrderQuantity) > 200
ORDER BY total_vendu DESC;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. La condition porte sur un total (`SUM`), donc elle va dans le `HAVING`, pas dans le `WHERE`.

Essayez maintenant de la mettre dans le `WHERE`, et faites-le correctement : déplacez la ligne `HAVING SUM(f.OrderQuantity) > 200` au-dessus du `GROUP BY` et remplacez le mot `HAVING` par `WHERE`. Le moteur répond `WHERE clause cannot contain aggregates!`, et c'est exactement ce qu'explique la théorie sur l'ordre d'exécution : au moment du `WHERE`, aucune somme n'a encore été calculée.

Si vous vous contentez de changer le mot sans déplacer la ligne, vous obtiendrez une autre erreur, `syntax error at or near "WHERE"`, qui ne dit rien de l'ordre d'exécution : elle dit seulement qu'un `WHERE` ne se met pas après un `GROUP BY`. Les deux erreurs sont instructives, mais ce n'est pas la même leçon.

Portabilité — Cette requête est la version OLAP, écrite sur l'entrepôt. Elle ne doit rien à DuckDB : `JOIN`, `GROUP BY`, `HAVING` et `ORDER BY` s'écrivent ainsi sur tous les moteurs, du plus petit à Snowflake. La même question se pose sur la version OLTP de la base, dans PostgreSQL, où les ventes vivent dans des tables normalisées : la Partie 3 la réécrit à titre de comparaison, avec les mêmes mots-clés et d'autres tables.

2.3 — `LEFT JOIN`, `FULL OUTER JOIN` et `USING`

Théorie : `Theorie_01_SQL.md`, section 7.

Objectif : tous les produits, avec le nom de leur sous-catégorie s'ils en ont une.

```sql
SELECT p.EnglishProductName             AS produit,
       s.EnglishProductSubcategoryName  AS sous_categorie
FROM dbo.DimProduct AS p
LEFT JOIN dbo.DimProductSubcategory AS s
       ON s.ProductSubcategoryKey = p.ProductSubcategoryKey
ORDER BY p.EnglishProductName;
```

Observez. `DimProduct`, écrite dans le `FROM`, est la table de gauche ; `DimProductSubcategory`, écrite après le `LEFT JOIN`, est la table de droite. Chaque ligne du résultat est donc un produit, prolongé à droite par sa sous-catégorie. Les produits sans sous-catégorie apparaissent quand même, avec `sous_categorie` à `NULL` : la moitié droite de leur ligne est restée vide. Remplacez `LEFT JOIN` par `JOIN` : ces produits disparaissent.

Faites maintenant l'expérience du piège. Ajoutez `WHERE s.EnglishProductSubcategoryName LIKE 'M%'` à la version `LEFT JOIN`, pour ne garder que les sous-catégories dont le nom commence par M (`LIKE` est présenté au Cours 02, section 2), et comptez les lignes. Placez-le avant le `ORDER BY`, sans quoi le moteur refusera la requête pour une raison qui n'a rien à voir avec le piège. Les produits sans sous-catégorie viennent de disparaître à nouveau, alors que vous n'avez pas touché à la jointure : le `WHERE` a rejeté leurs `NULL`. Déplacez la condition du `WHERE` vers le `ON`, en la préfixant par `AND`, et ils reviennent.

Passez au rapprochement des deux côtés. Le `FULL OUTER JOIN` garde à la fois les ventes sans produit et les produits sans vente :

```sql
SELECT COUNT(*)            AS lignes,
       COUNT(f.ProductKey) AS lignes_de_vente,
       COUNT(p.ProductKey) AS lignes_avec_produit
FROM dbo.FactInternetSales AS f
FULL OUTER JOIN dbo.DimProduct AS p ON p.ProductKey = f.ProductKey;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. 60 846 lignes : les 60 398 ventes, plus 448 produits jamais vendus en ligne. Le troisième compte égale le premier, donc aucune vente ne cite un produit inconnu. Une seule requête a vérifié les deux sens.

Réécrivez enfin la jointure du début avec `USING`, et regardez la clé :

```
SELECT ProductSubcategoryKey, p.EnglishProductName, s.EnglishProductSubcategoryName
FROM dbo.DimProduct AS p
LEFT JOIN dbo.DimProductSubcategory AS s USING (ProductSubcategoryKey)
ORDER BY p.EnglishProductName
LIMIT 5;
```

Observez. `ProductSubcategoryKey` se nomme sans préfixe, parce que `USING` n'en garde qu'une colonne ; Adjustable Race, sans sous-catégorie, y porte `NULL`. Remplacez `USING (ProductSubcategoryKey)` par `ON s.ProductSubcategoryKey = p.ProductSubcategoryKey` sans toucher au `SELECT` : DuckDB répond `Ambiguous reference to column name "ProductSubcategoryKey"`, parce que la clé existe maintenant deux fois, une par table.

2.4 — `NULL`, `COALESCE` et `IS DISTINCT FROM`

Théorie : `Theorie_01_SQL.md`, section 8.

Objectif : compter les lignes sans prix, puis afficher une valeur de remplacement.

```
-- Combien de lignes n'ont pas de prix de liste ?
SELECT COUNT(*) FROM dbo.DimProduct WHERE ListPrice IS NULL;

-- La MEME question, mais ecrite avec « = NULL » (piege classique) :
SELECT COUNT(*) FROM dbo.DimProduct WHERE ListPrice = NULL;

-- Les totaux sautent les NULL :
SELECT COUNT(*) AS nb_lignes, COUNT(ListPrice) AS nb_avec_prix
FROM dbo.DimProduct;

-- Afficher chaque produit avec sa taille, ou « sans taille » a defaut :
SELECT EnglishProductName, COALESCE(Size, 'sans taille') AS taille
FROM dbo.DimProduct
ORDER BY EnglishProductName;
```

Observez. `IS NULL` donne 211 et la version `= NULL` renvoie zéro, parce qu'elle ne teste rien. C'est le piège numéro un de `NULL`, et ce qui le rend redoutable, c'est que zéro est une réponse plausible. Le troisième compte dit la même chose autrement : 606 lignes, dont 395 seulement ont un prix.

Comparez maintenant à une valeur, avec et sans les `NULL` :

```
-- Les produits qui ne sont pas de taille M, deux fois :
SELECT COUNT(*) FROM dbo.DimProduct WHERE Size <> 'M';
SELECT COUNT(*) FROM dbo.DimProduct WHERE Size IS DISTINCT FROM 'M';
```


© 2026 iAlexMG. Tous droits réservés.

Observez. 285 contre 592. Les 307 produits sans taille ne sont pas de taille M, mais `<>` les écarte, parce que `NULL <> 'M'` vaut inconnu. `IS DISTINCT FROM` les compte : avec les 14 produits de taille M, on retombe sur les 606 lignes.

2.5 — « NA » : une absence déguisée en valeur

Théorie : `Theorie_01_SQL.md`, section 9.

Objectif : voir qu'une colonne peut n'avoir aucun `NULL` et manquer quand même d'information.

Commencez par le réflexe à prendre devant toute colonne que vous ne connaissez pas : regarder ses valeurs distinctes.

```sql
SELECT Color, COUNT(*) AS nb
FROM dbo.DimProduct
GROUP BY Color
ORDER BY nb DESC;
```

Observez. Aucune ligne `NULL`, mais une « couleur » `NA` en tête, avec 254 lignes. Refaites maintenant sur cette colonne les tests de la section 2.4 :

```sql
SELECT COUNT(*) FROM dbo.DimProduct WHERE Color IS NULL;

SELECT COUNT(*)                   AS nb_lignes,
       COUNT(Color)               AS nb_avec_couleur,
       COUNT(NULLIF(Color, 'NA')) AS nb_vraiment_colorees
FROM dbo.DimProduct;
```

Observez. `IS NULL` répond zéro, et `COUNT(Color)` compte les 606 lignes : pour le moteur, « NA » est une couleur comme une autre. Seul `NULLIF(Color, 'NA')`, qui transforme « NA » en `NULL`, retrouve les 352 lignes vraiment colorées.

La conséquence se voit dès la première analyse :

```sql
SELECT p.Color,
       COUNT(*)           AS lignes_vendues,
       SUM(f.SalesAmount) AS ventes
FROM dbo.FactInternetSales AS f
JOIN dbo.DimProduct        AS p ON p.ProductKey = f.ProductKey
GROUP BY p.Color
ORDER BY lignes_vendues DESC;
```

Observez. « NA » arrive en tête, avec 28 919 lignes de vente sur 60 398, presque la moitié, mais seulement 435 117 $ sur 29,4 millions. Ce sont des accessoires bon marché : bidons, chambres à air, trousses de réparation. Un tableau de bord qui titrerait « NA, notre couleur la plus vendue » serait juste au sens de SQL et absurde au sens du métier.

Dernière vérification : la convention n'est même pas la même d'une colonne à l'autre.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT Size, SizeRange, COUNT(*) AS nb
FROM dbo.DimProduct
WHERE Size IS NULL OR SizeRange = 'NA'
GROUP BY Size, SizeRange;
```

Observez. Le `OR` garde une ligne dès que l'une des deux conditions est vraie (Cours 02, section 2). Une seule ligne de résultat : les 307 lignes sans taille ont `Size` à `NULL` et `SizeRange` à « NA ». La même absence, écrite de deux façons dans deux colonnes voisines. La section 9 de `Theorie_01_SQL.md` explique pourquoi un entrepôt fait ce choix, et ce qu'il coûte.

Partie 3 — Un regard vers l'OLTP

Théorie : `Theorie_01_SQL.md`, section 3.

Rien à exécuter ici. C'est un avant-goût du Cours 02, où le double chemin commence pour de bon et où vous taperez vraiment les deux versions. Si vous avez déjà installé PostgreSQL au Cours 00, rien ne vous empêche d'essayer.

La question 2.2, « quels produits se vendent le plus », s'écrit presque pareil sur la version OLTP normalisée. Là, les ventes ne sont pas une table de faits mais une table de lignes de commande, `sales.salesorderdetail`, reliée aux produits par `productid` :

```sql
-- Version OLTP (PostgreSQL) de la requete 2.2, a titre de comparaison
SELECT p.name, SUM(d.orderqty) AS total_commande
FROM sales.salesorderdetail AS d
JOIN production.product      AS p ON p.productid = d.productid
GROUP BY p.name
HAVING SUM(d.orderqty) > 1000
ORDER BY total_commande DESC;
```

Le même SQL, avec `JOIN`, `GROUP BY` et `HAVING`. Seuls changent les noms (fait et dimension d'un côté, tables normalisées de l'autre), la clé de jointure, et la casse, en minuscules côté portage PostgreSQL. C'est tout le propos de la formation : un seul langage, deux mondes.

Le seuil, lui, passe de 200 à 1 000, et la raison tient aux données, pas au langage. `FactInternetSales` ne garde que les ventes en ligne : 60 398 lignes d'une unité chacune. `sales.salesorderdetail` contient toutes les commandes : les mêmes 60 398 lignes en ligne, plus 60 919 lignes de revendeurs, qui achètent en gros et totalisent à elles seules 214 516 unités. Plus de volume, d'où un seuil plus haut.

Un autre écart vous attend là-bas, qui prolonge la section 2.5 : dans `production.product`, la couleur absente est un vrai `NULL`, et c'est le prix des pièces détachées qui porte une sentinelle, un 0.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_01_SQL.md`.

© 2026 iAlexMG. Tous droits réservés.

Et ensuite ?

Votre entrepôt resservira à tous les cours suivants : rien à réinstaller. Le Cours 02 fait le tour du langage lui-même (filtres, conditions, types, texte, dates, agrégats) et ouvre le double chemin : deux laboratoires par module, un sur chaque terrain. C'est le moment de finir l'installation de PostgreSQL si vous l'aviez remise à plus tard.

© 2026 iAlexMG. Tous droits réservés.